



Project Report on
Support Vector Machine for Cost-effective Feature Selection

IEDA 3010 Prescriptive Analytics

19th December 2024

Group 21
WONG Man Chun, Matthew
TANG Cheuk Him, Thomas
XIONG Zelin
TIN Yiu Tsun, Jason
WEN Meng Qian, Velma

Introduction

The problem of high-dimensional data has grown in importance due to the quick development of data collecting technology, frequently resulting in problems like the curse of dimensionality. Efficient feature selection is essential for improving computational efficiency and model predictability, but conventional approaches usually ignore the related feature acquisition costs. In industries like healthcare, where medical testing might have large financial ramifications, this control is especially important. The goal of this project is to create affordable feature selection models that take into account the financial limitations of practical applications while simultaneously increasing classification accuracy.

This project presents Group Feature Selection models (GFS-CESVM1 and GFS-RCESVM1) that handle both individual and group feature selection within a budgetary framework, building on earlier work with Cost-Effective ℓ_1 -Norm Support Vector Machine (CESVM1) and its robust variant (RCESVM1). We provide a Branch-Cut-and-Price (BCP) algorithm that is intended to maximize performance while controlling costs in order to address the computational complexity seen in Mixed Integer Linear Programming (MILP) formulations. This project aims to provide workable strategies for economical feature selection in machine learning by testing on a variety of datasets, ultimately enhancing decision-making across a range of domains.

Background and Research Questions

Chronic Kidney Disease (CKD) is a long-term condition characterized by a gradual decline in kidneys' ability, defined by kidney damage or a reduced glomerular filtration rate (GFR) lasting three months or more. CKD is classified into five stages based on kidney function, ranging from mild (stage 1) to end-stage renal disease (stage 5). Globally, 1.2 million (95% UI) individuals died from CKD in 2017, and 697.5 million (95% UI 649.2 to 752.0 million) cases of all-stage CKD were reported.¹ In the early stages, CKD may not present noticeable symptoms, but as it progresses, individuals may experience fatigue, swelling, changes in urination, and high blood pressure. Hence the early detection of CKD is crucial as it enables timely interventions that can slow disease progression and prevent complications. Additionally, it helps reduce healthcare costs by minimizing the need for advanced treatments and hospitalizations associated with end-stage renal disease.

With traditional machine learning models like CESVM1 and RCESVM1 exist concerns like overfitting, model complexity and generalization, these models may not be flexible enough to handle the complex data. Hence our project aims to identify the comparison between our model with other mentioned models when detecting a CKD patient in terms of accuracy and effectiveness and determine the best model, categorize patients into CKD and non-CKD groups with a high level of accuracy compared to traditional ML model, seek out the most relevant clinical features and remove the irrelevant or useless features. Educating excessive costs and resource limitations due to High-performing features that may lead to impractical solutions.

¹ Kidney Disease Collaboration, G. C. (2020). Global, regional, and national burden of chronic kidney disease, 1990–2017: A systematic analysis for the Global Burden of Disease Study 2017. *Lancet (London, England)*, 395(10225), 709. [https://doi.org/10.1016/S0140-6736\(20\)30045-3](https://doi.org/10.1016/S0140-6736(20)30045-3)

Model Setup

Our research addresses a critical gap in standard SVM methodologies by incorporating cost considerations into the feature selection process. By extending the ℓ_1 -SVM with a budget constraint, we ensure that our model not only maintains high classification accuracy but also selects the most informative and cost-effective features within a predefined budget. This approach, termed CESVM1, promotes sensible decision-making in real-world applications, ultimately leading to more practical and optimal solutions.

The introduction of binary decision variables v_j enables a clear mechanism for selecting features while adhering to the budget constraint B . By ensuring that the total cost of selected features remains within this budget, we enhance the practicality of our model. Additionally, the activation and deactivation of the weight vector based on selected features allow for a more tailored approach to feature optimization. It is crucial to set the bounds M_j appropriately to maintain both optimality and feasibility of the solution. This structured approach ultimately leads to a more efficient and effective feature selection process.

$$\begin{aligned} \text{Minimize} \quad & \sum_{j=1} (w^+_j + w^-_j) + C \sum_{i=1} \xi_i \quad (3.11) \\ \text{w}^+, \text{w}^-, \text{v}, \xi, \text{b} \end{aligned}$$

$$\text{s.t. } y_i (\sum ((w^+_j - w^-_j) \cdot x_{ij}) + b) \geq 1 - \xi_i, \quad \forall i, \quad (3.12)$$

$$\sum_{j=1} c_j v_j \leq B, \quad (3.13)$$

$$w^+_j + w^-_j \leq M_j v_j, \quad \forall j, \quad (3.14)$$

$$\xi_i \geq 0, \quad \forall i, \quad (3.15)$$

$$w^+_j, w^-_j \geq 0, \quad \forall j, \quad (3.16)$$

$$v_j \in \{0, 1\}, \quad \forall j, \quad (3.17)$$

Robust optimization plays a crucial role in ensuring the practicality of solutions in the presence of uncertainty. By building on the foundations laid by previous models, we introduce the robust and cost-effective model, RCESVM1, which effectively balances flexibility and computational efficiency. This model accounts for uncertain feature costs through the parameter Γ , allowing for a controlled degree of conservatism. As Γ approaches the total number of uncertain features, RCESVM1 becomes increasingly conservative, while retaining the optimal characteristics of CESVM1 when $\Gamma=0$. This approach not only enhances the robustness of feature selection but also ensures cost-effectiveness, making it a valuable tool for real-world applications. CESVM1 can be reformulated as RCESVM1 by transforming Eq. (3.13) as follows:

$$\sum_{j=1}^J c_j v_j + \max_{\{S: S \subseteq J, |S| \leq \Gamma\}} \left\{ \sum_{j \in S} \hat{c}_j v_j \right\} \leq B$$

The feasibility of the solution is safeguarded against the cost uncertainty of Γ features through Eq. 3.19. While this term introduces complexity to RCESVM1 by compromising its linearity, it has been demonstrated that $\beta(v, \Gamma)$ can be transformed into a linear optimization model without sacrificing generality. For a given vector v^* , the protection function $\beta(v^*, \Gamma)$ can be defined as Eq (3.20-3.22)

$$\beta(v^*, \Gamma) = \max_{\{S: S \subseteq J, |S| \leq \Gamma\}} \left\{ \sum_{j \in S} \hat{c}_j v_j^* \right\}. \quad (3.19)$$

$\beta(v^*, \Gamma)$ is equivalent to the objective function of the following linear optimization problem.

$$\beta(v^*, \Gamma) = \text{maximize } \sum_{j \in J} \hat{c}_j v_j^* z_j \quad (3.20)$$

$$\text{s.t. } \sum_{j \in J} z_j \leq \Gamma, \quad (3.21)$$

$$0 \leq z_j \leq 1, \quad \forall j \in J. \quad (3.22)$$

The optimal solution of Eqs. (3.20)–(3.22) corresponds to the selected subset S with the associated cost function $\sum_{j \in J} \hat{c}_j v_j^*$ as shown in Eq. (3.19). Consequently, the dual of Eqs. (3.20)–(3.22) can be defined as Eq (3.23-3.26)

$$\text{Minimize } \sum_{j \in J} p_j + z\Gamma \quad (3.23)$$

$$\text{s.t. } z + p_j \geq \hat{c}_j v_j^*, \quad \forall j \in J, \quad (3.24)$$

$$p_j \geq 0, \quad \forall j \in J, \quad (3.25)$$

$$z \geq 0. \quad (3.26)$$

Realization

standard SVM

```

model = Model("L1_SVM")
n_features = X_train.shape[1]
w_plus = model.addVars(n_features, lb=0, name="w_plus")
w_minus = model.addVars(n_features, lb=0, name="w_minus")
xi = model.addVars(len(y_train), lb=0, name="xi")
b = model.addVar(lb = 0, name = "b")
C = 0.1

model.setObjective(quicksum(w_plus[j] + w_minus[j] for j in range(n_features)) + C * quicksum(xi[i] for i in range(len(y_train))), GRB.MINIMIZE)

for i in range(len(y_train)):
    dot_product = quicksum((w_plus[j] - w_minus[j]) * X_train.iloc[i, j] for j in range(n_features))
    model.addConstr(y_train.iloc[i] * (dot_product + b) >= 1 - xi[i])

model.optimize()

if model.status == GRB.OPTIMAL:
    print("Optimal Weights:")
    for j in range(n_features):
        print(f"Feature {j}: w+ = {w_plus[j].X}, w- = {w_minus[j].X}, b : {b.X}")

l1weights_data = {
    'w+': [w_plus[j].X for j in range(n_features)],
    'w-': [w_minus[j].X for j in range(n_features)],
}
l1SVM_b = b.X

l1weights_df = pd.DataFrame(l1weights_data)
print(l1weights_df)

```

Weighting of different features under different C:

```

Optimal Weights:(C:0.1)(b:16.192608761781237)
Feature 0: w+ = 0.0, w- = 0.000803361109149325
Feature 1: w+ = 0.014446682513734642, w- = 0.0
Feature 2: w+ = 0.0, w- = 0.0
Feature 3: w+ = 0.6462305084065333, w- = 0.0
Feature 4: w+ = 0.0, w- = 0.0
Feature 5: w+ = 0.0, w- = 0.0
Feature 6: w+ = 0.0, w- = 0.0
Feature 7: w+ = 0.0, w- = 0.0
Feature 8: w+ = 0.0, w- = 0.0
Feature 9: w+ = 0.01443625450172386, w- = 0.0
Feature 10: w+ = 0.0, w- = 0.007548555051715776
Feature 11: w+ = 0.342847477288618, w- = 0.0
Feature 12: w+ = 0.0, w- = 0.054656769321049466
Feature 13: w+ = 0.0, w- = 0.0
Feature 14: w+ = 0.0, w- = 0.41167907860806696
Feature 15: w+ = 0.0, w- = 0.1336602779044024
Feature 16: w+ = 0.0, w- = 2.6117190098534147e-05
Feature 17: w+ = 0.0, w- = 0.06054813476428543
Feature 18: w+ = 0.09604875573443926, w- = 0.0
Feature 19: w+ = 0.1736246923267261, w- = 0.0
Feature 20: w+ = 0.0, w- = 0.0
Feature 21: w+ = 0.0, w- = 0.0
Feature 22: w+ = 0.0, w- = 0.0
Feature 23: w+ = 0.0, w- = 0.0

```

```

Optimal Weights:(C:0.01)(b:14.263547746927976)
Feature 0: w+ = 0.0002338390816405102, w- = 0.0
Feature 1: w+ = 0.013250481121343869, w- = 0.0
Feature 2: w+ = 0.0, w- = 0.0
Feature 3: w+ = 0.0, w- = 0.0
Feature 4: w+ = 0.0, w- = 0.0
Feature 5: w+ = 0.0, w- = 0.0
Feature 6: w+ = 0.0, w- = 0.0
Feature 7: w+ = 0.0, w- = 0.0
Feature 8: w+ = 0.0, w- = 0.0
Feature 9: w+ = 0.013727397597701424, w- = 0.0
Feature 10: w+ = 0.008208355569830529, w- = 0.0
Feature 11: w+ = 0.0, w- = 0.0
Feature 12: w+ = 0.0, w- = 0.07152308990127752
Feature 13: w+ = 0.0, w- = 0.0
Feature 14: w+ = 0.0, w- = 0.07205867394857379
Feature 15: w+ = 0.0, w- = 0.17137495693335472
Feature 16: w+ = 9.12948118349917e-05, w- = 0.0
Feature 17: w+ = 0.0, w- = 0.0
Feature 18: w+ = 0.0, w- = 0.0
Feature 19: w+ = 0.0, w- = 0.0
Feature 20: w+ = 0.0, w- = 0.0
Feature 21: w+ = 0.0, w- = 0.0
Feature 22: w+ = 0.0, w- = 0.0
Feature 23: w+ = 0.0, w- = 0.0

```

Optimal Weights:(C:1)(b:16.99873780787285)
 Feature 0: w+ = 0.0, w- = 0.007644838788542326
 Feature 1: w+ = 0.02175915898163083, w- = 0.0
 Feature 2: w+ = 0.0, w- = 0.0
 Feature 3: w+ = 1.9565604064573825, w- = 0.0
 Feature 4: w+ = 0.4503901537881975, w- = 0.0
 Feature 5: w+ = 0.0, w- = 0.13616809856814568
 Feature 6: w+ = 0.0, w- = 0.6151184553415557
 Feature 7: w+ = 0.0, w- = 0.0
 Feature 8: w+ = 0.0, w- = 0.0
 Feature 9: w+ = 0.014909658047809653, w- = 0.0
 Feature 10: w+ = 0.0, w- = 0.024470511036771676
 Feature 11: w+ = 0.538740716962074, w- = 0.0
 Feature 12: w+ = 0.0, w- = 0.034174869390951
 Feature 13: w+ = 0.0, w- = 0.0
 Feature 14: w+ = 0.0, w- = 0.3874604399866232
 Feature 15: w+ = 0.0, w- = 0.14537472146999875
 Feature 16: w+ = 6.780485578826534e-05, w- = 0.0
 Feature 17: w+ = 0.0, w- = 0.5066033854967347
 Feature 18: w+ = 0.7877928672538421, w- = 0.0
 Feature 19: w+ = 0.4189619972830371, w- = 0.0
 Feature 20: w+ = 0.0, w- = 0.0
 Feature 21: w+ = 0.0, w- = 0.6265196484366151
 Feature 22: w+ = 0.3329933572913443, w- = 0.0
 Feature 23: w+ = 0.0, w- = 0.0

Optimal Weights:(C:10)(b:36.06463360830768)
 Feature 0: w+ = 0.0, w- = 0.00954837700645012
 Feature 1: w+ = 0.021511329319590836, w- = 0.0
 Feature 2: w+ = 0.0, w- = 0.0
 Feature 3: w+ = 2.9260414470844993, w- = 0.0
 Feature 4: w+ = 0.15462892797094208, w- = 0.0
 Feature 5: w+ = 0.0, w- = 1.5928077626329984
 Feature 6: w+ = 0.0, w- = 0.0
 Feature 7: w+ = 0.0, w- = 0.0
 Feature 8: w+ = 0.0, w- = 0.0
 Feature 9: w+ = 0.012226688382835272, w- = 0.0
 Feature 10: w+ = 0.0, w- = 0.03799441063998152
 Feature 11: w+ = 0.7416781577177639, w- = 0.0
 Feature 12: w+ = 0.0, w- = 0.09718317414892731
 Feature 13: w+ = 0.0, w- = 0.0
 Feature 14: w+ = 0.0, w- = 0.7455456619376578
 Feature 15: w+ = 0.0, w- = 0.10230066101244326
 Feature 16: w+ = 9.052981562570912e-05, w- = 0.0
 Feature 17: w+ = 0.0, w- = 1.5467576675823296
 Feature 18: w+ = 0.007326275747548753, w- = 0.0
 Feature 19: w+ = 2.155684870149107, w- = 0.0
 Feature 20: w+ = 0.0, w- = 0.0
 Feature 21: w+ = 0.0, w- = 0.7452896509639048
 Feature 22: w+ = 0.44290575781949587, w- = 0.0
 Feature 23: w+ = 0.4491520233818907, w- = 0.0

The performance of the SVM with different C:

C = 0.1

Accuracy: 0.9775

Precision: 0.9839357429718876

Recall (Sensitivity): 0.98

F1-Score: 0.9819639278557114

C = 0.01

Accuracy: 0.94

Precision: 0.9669421487603306

Recall (Sensitivity): 0.936

F1-Score: 0.9512195121951219

C = 1

Accuracy: 0.995

Precision: 0.996

Recall (Sensitivity): 0.996

F1-Score: 0.996

C = 10

Accuracy: 0.9925

Precision: 0.9959839357429718

Recall (Sensitivity): 0.992

F1-Score: 0.9939879759519038

Solution

In our analysis, we implemented robust techniques to enhance the predictive capabilities of our model for Chronic Kidney Disease (CKD). The final results demonstrated a notable 20% increase in accuracy, which was accompanied by an improved precision rate. This advancement is crucial as it significantly reduces the incidence of false positives, thereby minimizing the risks of misdiagnosis. Consequently, this not only alleviates unnecessary patient anxiety but also mitigates the costs associated with confirming the absence of CKD.

A thorough examination of data from CKD patients revealed a recall rate of 1, indicating that our model successfully detected all cases of CKD without missing any. Additionally, the F1 score—a critical measure that represents the harmonic mean of recall and precision—showed substantial improvement, further confirming the enhanced accuracy of our model following the robust adjustments.

Recommendations

Given the increased accuracy of our model, we recommend that patients may not require extensive feature checks to ensure reliability. This approach leads to lower healthcare costs and a reduction in time spent on unnecessary tests.

Furthermore, we employed a grid search methodology for data comparison, allowing us to utilize more grid points. This strategy enabled a more thorough exploration of possibilities, contributing to the accuracy of our results.

We also identified key features that emerged as significant in our analysis. Features with higher weighting indicate greater importance in predicting CKD, which provides valuable insights for future assessments.

Conclusion

In summary, our comparative analysis utilizing L1 norm SVM, CESVM, and RCESVM has demonstrated a model capable of efficient predictions under cost considerations. This framework not only supports improved decision-making in real-world applications but also ensures that the selected features align with both performance objectives and budgetary limitations. By integrating cost-effectiveness into the feature selection process, we can enhance healthcare outcomes while managing resources effectively.

Reference

Kidney Disease Collaboration, G. C. (2020). Global, regional, and national burden of chronic kidney disease, 1990–2017: A systematic analysis for the Global Burden of Disease Study 2017. *Lancet (London, England)*, 395(10225), 709. [https://doi.org/10.1016/S0140-6736\(20\)30045-3](https://doi.org/10.1016/S0140-6736(20)30045-3)